

Forward without forward

Contents

- [Motivation](#)
- [Proposal](#)

Motivation

Consider the following example:

```
template <class X, class Y>
decltype(auto) foo(X&& x, Y&& y) {
    return std::forward<X>(x)(std::forward<Y>(y));
}
```

It's a simple function, but it takes some time to figure out what's actually going on. It's even more awkward when we consider the same function in lambda form:

```
auto bar = [](auto&& x, auto&& y) {
    return std::forward<decltype(x)>(x)(std::forward<decltype(y)>(y));
};
```

This verbosity and lack of clarity lead many people to actually use a *macro* to make forwarding shorter:

```
#define FWD(x) std::forward<decltype(x)>(x)

template <class X, class Y>
decltype(auto) foo(X&& x, Y&& y) {
    return FWD(x)(FWD(y));
}

auto bar = [](auto&& x, auto&& y) {
    return FWD(x)(FWD(y));
};
```

That is substantially clearer, but are we really going to suggest that people use a macro?! Let's not. Forwarding references even got their name from the fact that they are intended to be `std::forward()`-ed, so it's awkward when the expected and typical usage is just so very verbose. The body of the lambda with `std::forward` is 73 characters, the body with `FWD` is 23. Do those extra 50 characters add any value to anybody?

A second problem with forwarding is its lack of nice interaction with lambda capture. We have a simple way of capturing by copy and by reference. But we do not have a simple way of capturing by decay-copy, which is quite common when we want the lambda to acquire ownership of the relevant objects. Consider trying to write a function that delays invoking a particular function. Currently, that might be:

```
template <class F, class... Args>
auto delay_invoke(F&& f, Args&&... args) {
    return [f=std::forward<F>(f), tup=std::make_tuple(std::forward<Args>(args)...)] {
        return std::apply(std::forward<F>(f), std::move(tup));
    };
}
```

There's just no simple way to capture `f` or `args...` in this example, so almost everything in this function is fighting with the language to express a simple idea. Compare this to an example where we don't care about forwarding, the kind of example that might appear in talks or slides:

```
template <class F, class... Args>
auto delay_invoke(F f, Args... args) {
    return [=]() { return std::invoke(f, args...); };
}
```

Code that forwards is just so much more verbose than code that doesn't. But forwarding in itself doesn't justify the verbosity or resulting complexity. Unlike `std::move` and `std::ref` which are used to do non-typical things and deserve to be visible markers for code readability and understading, `std::forward` is much more typical in these contexts and does not need as much of a sign post. We can do better.

Proposal

This paper would like to see a shorter way to forward arguments and proposes non-overloadable unary `operator>>`, where `>>expr`

is defined as `static_cast<decltype(expr)&&>(expr)`. Furthermore, the symbol `>>` is to be usable as another form of capture in lambda expressions to decay-copy.

This addition would make it easier to write and read code that uses forwarding:

C++17

This proposal

<pre>template <class X, class Y> decltype(auto) foo(X&& x, Y&& y) { return std::forward<X>(x)(std::forward<Y>(y)); } auto bar = [](auto&& x, auto&& y) { return std::forward<decltype(x)>(x)(std::forward<decltype(y)>(y)); };</pre>	<pre>template <class X, class Y> decltype(auto) foo(X&& x, Y&& y) { return >>x(>>y); } auto bar = [](auto&& x, auto&& y) { return >>x(>>y); };</pre>
<pre>template <class F, class... Args> auto delay_invoke(F&& f, Args&&... args) { return [f=std::forward<F>(f), tup=std::make_tuple(std::forward<Args>(args)...)] { return std::apply(std::forward<F>(f), std::move(tup)); }; }</pre>	<pre>template <class F, class... Args> auto delay_invoke(F&& f, Args&&... args) { return [>>] { return std::invoke(>>f, >>args...); }; }</pre>

This proposal will not deprecate `std::forward()`, as there is another functionality of this function template that is quite important: forwarding expressions to a *different* type. This isn't common, but is more important to highlight as different for simple forwarding. This extension allows for making simple forwarding look simple, while making complex forwarding look complex.

Unary `>>` is ill-formed today, and that token cannot appear in lambda capture in this way, so this is purely a language extension.